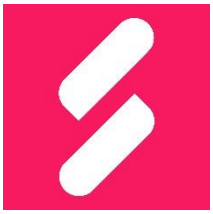
UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA



Socially – social media

Name: Ernst Robert
Group: 30236

Table of Contents

Deliverable 1 ..... **3**

Project Specification **3**

Functional Requirements **4**

Use Case Model 1 ..... **4**

Use Cases Identification 4

UML Use Case Diagrams **5**

Supplementary Specification **7**

Non-functional Requirements 7

Design Constraints 9

Glossary **9**

Deliverable 2 ..... **10**

Domain Model **10**

Architectural Design **16**

Conceptual Architecture: 16

Package Design: 17

Component and Deployment Diagram 17

Deliverable 3 ..... **19**

Design Model **19**

Dynamic Behavior 19

Class Diagram 21

Data Model **27**

System Testing **28**

Future Improvements **30**

Conclusion **31**

Bibliography **32**

Deliverable 1

Project Specification

1. Overview

This project aims to develop a social media application inspired by Snapchat's core functionalities, allowing users to share moments through pictures and messages instantly. The application will exclude location tracking, reels, and video/call features, focusing on providing a seamless and user-friendly experience for sharing media and messaging.

2. Technology Stack

- **API Gateway**: A custom-built API gateway that routes requests to the appropriate microservices, leveraging the reactive programming model for optimal performance.
- **Security**: The application leverages OAuth2 Authorization Code flow integrated into the API Gateway, ensuring a secure and standardized approach to authentication and authorization across all microservices.
- **Microservices**: Accounts, Messaging, Media, Connections, and Notifications, all designed to be reactive and efficiently handle asynchronous data streams.
- **Backend**: Developed using Spring Boot and Spring WebFlux to enable a non-blocking, reactive microservices architecture. This approach enhances the application's scalability and responsiveness.
- **Database**: Postgres will be used for storing relational data, such as user profiles, friendships, and messaging metadata, and MongoDB for NoSQL data storage needs, particularly for storing unstructured data such as media files, logs, and notifications.
- **Media Storage:** Utilizes Google Firebase Cloud Storage for storing and serving media files (photos, videos, etc.).
- **Caching**: Custom cache implementation using Aspect-Oriented Programming to decouple cache cross-cutting concerns from business logic, ensuring efficient and scalable caching strategies across services.
- **Prometheus**: Key part of the Monitoring and Observability, is employed for gathering and storing metrics from the API gateway.
- **Grafana**: Utilized for visualizing the metrics collected by Prometheus. Custom dashboards are created to monitor vital application metrics such as request rates, errors, and system resource usage in real-time.

3. Architecture

The application adopts a microservices architecture, facilitating independent development, deployment, and scaling of its core functionalities. The custom API gateway serves as the entry point, directing requests to the relevant services, thereby ensuring efficient load distribution and service discovery.

Functional Requirements

1. **Accounts:**
    - Users can create, edit their accounts.
    - Users can login with any of username, phone number or email address.
    - Admins can do all of the users functionality + delete any accounts.
2. **Messaging:**
    - Users can send and receive messages in real-time via WebSockets.
    - Messages can be ephemeral.
3. **Media Sharing:**
    - Users can upload photos to their accounts, or through stories.
    - Users can share photos, through snaps or conversations.
4. **Connections:**
    - Users can add friends through usernames, phone numbers or email.
    - Provides functionality to accept, reject, or block connection requests**.**
5. **Notifications:**
    - Real-time notifications for new messages, friend requests, and other relevant activities.

Use Case Model 1

Use Cases Identification

Use-Cases:

- Register
- Update Account
- View Account
- Login
- Delete Account
- Send snap
- Open snap
- View stories
- Add story
- Delete story

- Add friend
- Delete friend
- Block friend
- Unblock friend
- Send message
- Delete message
- View conversations
- View messages
- View notifications

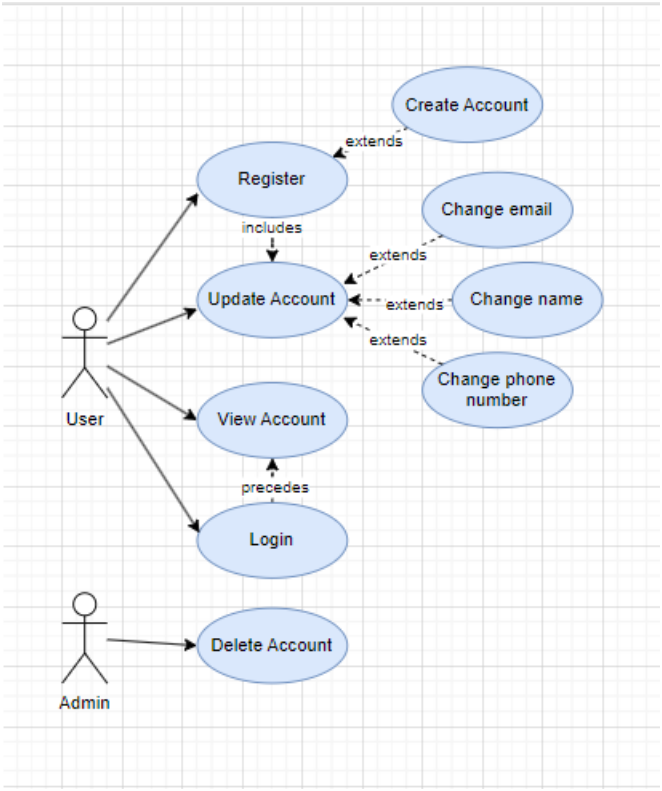
Level: High-Level

Primary Actor: User

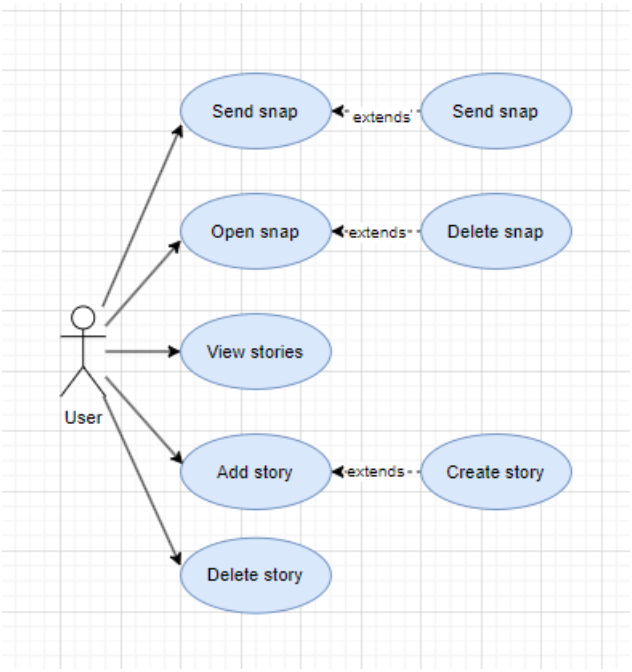
UML Use Case Diagrams

These are the UML Use Case Diagrams based on the identified use-cases, classified by the microservices:

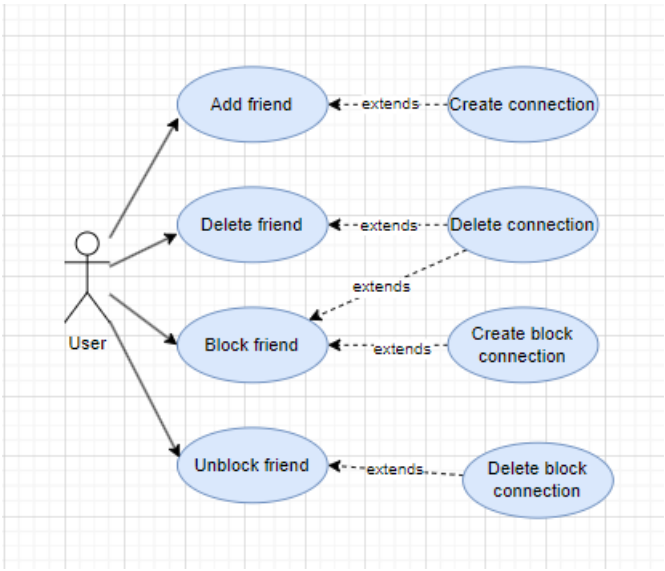
- **Accounts**:



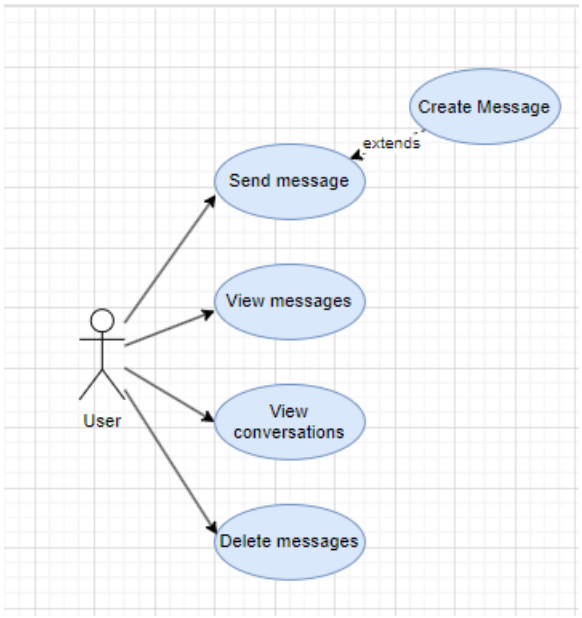
- **Media:**



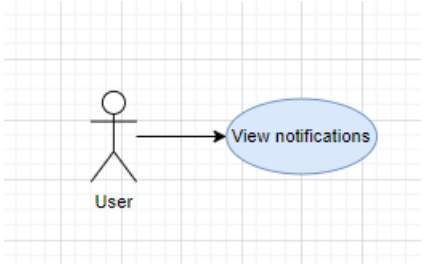
- **Connections:**



- **Messaging:**



- **Notifications:**



Supplementary Specification

Non-functional Requirements

1. **Performance**
   The system is designed to ensure high performance under varying loads, leveraging caching and a reactive architecture to meet user expectations for responsiveness and speed.
   - **Caching**: Implement custom caching strategies using Aspect-Oriented Programming (AOP) to minimize database access times and reduce load on the system. Effective caching of frequently accessed data ensures that the application responds quickly to user requests, improving the overall user experience.
   - **Reactive Architecture**: Utilize Spring WebFlux to build a non-blocking, reactive microservices architecture. This approach allows the application to handle large numbers of concurrent users and operations efficiently, by optimizing resource usage and improving scalability. The reactive model also enhances the application's responsiveness, as it can process requests and responses asynchronously.

2. **Scalability**

The application is built to scale seamlessly to accommodate growth in users and data, utilizing a microservices architecture and multiple databases to distribute load and enhance system capacity.

- **Microservices Architecture**: Each microservice (Accounts, Messaging, Media, Connections, Notifications) is independently scalable, allowing for targeted scaling of components based on demand. This modular approach enables efficient utilization of resources and simplifies updates and maintenance, contributing to the system's overall scalability.
- **Multiple Databases**: Use PostgreSQL for relational data management and MongoDB for NoSQL data requirements, ensuring that the system can handle diverse data types and structures efficiently. This multi-database strategy supports scalability by optimizing data storage and retrieval processes according to the specific needs of each service.
- **Cloud Storage for Media**: Integrate Firebase Cloud Storage for media files, offloading storage and bandwidth demands from the main infrastructure. This ensures that the system can scale to support increasing volumes of media content without compromising performance.

3. **Security**

The application implements robust security mechanisms, utilizing OAuth2 Authorization Code Flow Grant to protect resources and ensure that user data is securely accessed and transmitted.

- **OAuth2 Authorization Code Flow**: This secure authentication and authorization protocol provides tokens to clients after successfully authenticating the user and obtaining authorization. It adds an extra layer of security by requiring an authorization code exchange, which happens server-side. This mitigates the risk of token interception and ensures that sensitive information is only accessible to authenticated and authorized users.
- **Secure API Gateway**: The custom API gateway incorporates OAuth2 to manage and validate access tokens for incoming requests, serving as the first line of defense against unauthorized access to the system's internal services.

4. **Maintainability**

The system is designed for ease of maintenance, employing monitoring and observability tools to quickly identify and address issues, ensuring long-term reliability and ease of updates.

- **Monitoring and Observability**: Implement Prometheus for metric collection and Grafana for visualization, enabling real-time monitoring of the application's health and performance. This allows for the proactive identification of issues, understanding system behavior, and making informed decisions on optimizations, ensuring the system remains maintainable and up-to-date with minimal effort.
- **Log Management**: Use structured logging to capture and store logs in a centralized system, facilitating easy debugging and monitoring of the application's operational status. This aids in quick issue resolution and system maintenance.

Design Constraints

- Given that in the Http Client from API Gateway, we cannot put a body in a GET request, all GET requests must have the body in the path variable.
- Given that is not worth to have a caching system waiting on an asynchronous call, we'll add the caching at the Service layer, and we'll remove non-blocking actions in any Service.

Glossary

API Gateway: A server that acts as an intermediary for some of Microservices architecture's requests, routing them to the appropriate microservice. It also handles cross-cutting concerns such as security (via OAuth2), rate limiting, and metrics collection.

Aspect-Oriented Programming (AOP): A programming paradigm that aims to increase modularity by allowing the separation of cross-cutting concerns. It is used in the application for implementing caching mechanisms, thereby decoupling caching logic from business logic.

Authentication: The process of verifying the identity of a user or system. In the context of this application, it is achieved through OAuth2 Authorization Code Flow Grant, ensuring secure access to the application's resources.

Authorization: The process of determining if an authenticated user has permission to perform a specific action or access a particular resource within the application. Managed by OAuth2 in conjunction with the API Gateway.

Caching: Temporarily storing data in a local storage to reduce the number of calls to the database, improving the performance and scalability of the application. Custom caching strategies are implemented via AOP.

Grafana: An open-source platform for monitoring and observability, used to visualize metrics collected from the application's operations, including those gathered by Prometheus.

Microservices Architecture: A method of developing software applications as a suite of independently deployable, modular services, each running a unique process and communicating through lightweight mechanisms, typically an HTTP-based API.

MongoDB: A NoSQL database used for storing schema-less data, such as user profiles and messaging history, providing flexibility and scalability for handling large volumes of data that does not fit into a traditional relational database model.

OAuth2 Authorization Code Flow Grant: A secure authorization flow that authenticates users and authorizes applications to access user data without exposing their credentials, using an authorization server to issue tokens upon successful authentication.

PostgreSQL: An open-source relational database management system (RDBMS) used for storing and managing structured data with integrity constraints and complex queries.

Prometheus: An open-source system monitoring and alerting toolkit, used for collecting and processing metrics from microservices and the API gateway, facilitating real-time performance monitoring.

Reactive Architecture: A programming paradigm oriented around data streams and the propagation of change, enabling the development of non-blocking, asynchronous applications that are scalable and resilient. Implemented in this project using Spring WebFlux.

Spring WebFlux: A reactive stack in the Spring framework designed to build non-blocking, asynchronous web applications on the Reactive Streams specification.

Firebase Cloud Storage: A cloud service provided by Google Firebase for storing and serving user-generated content, such as photos and videos, allowing for scalable and secure media storage separate from the application's primary infrastructure.

Deliverable 2

Domain Model

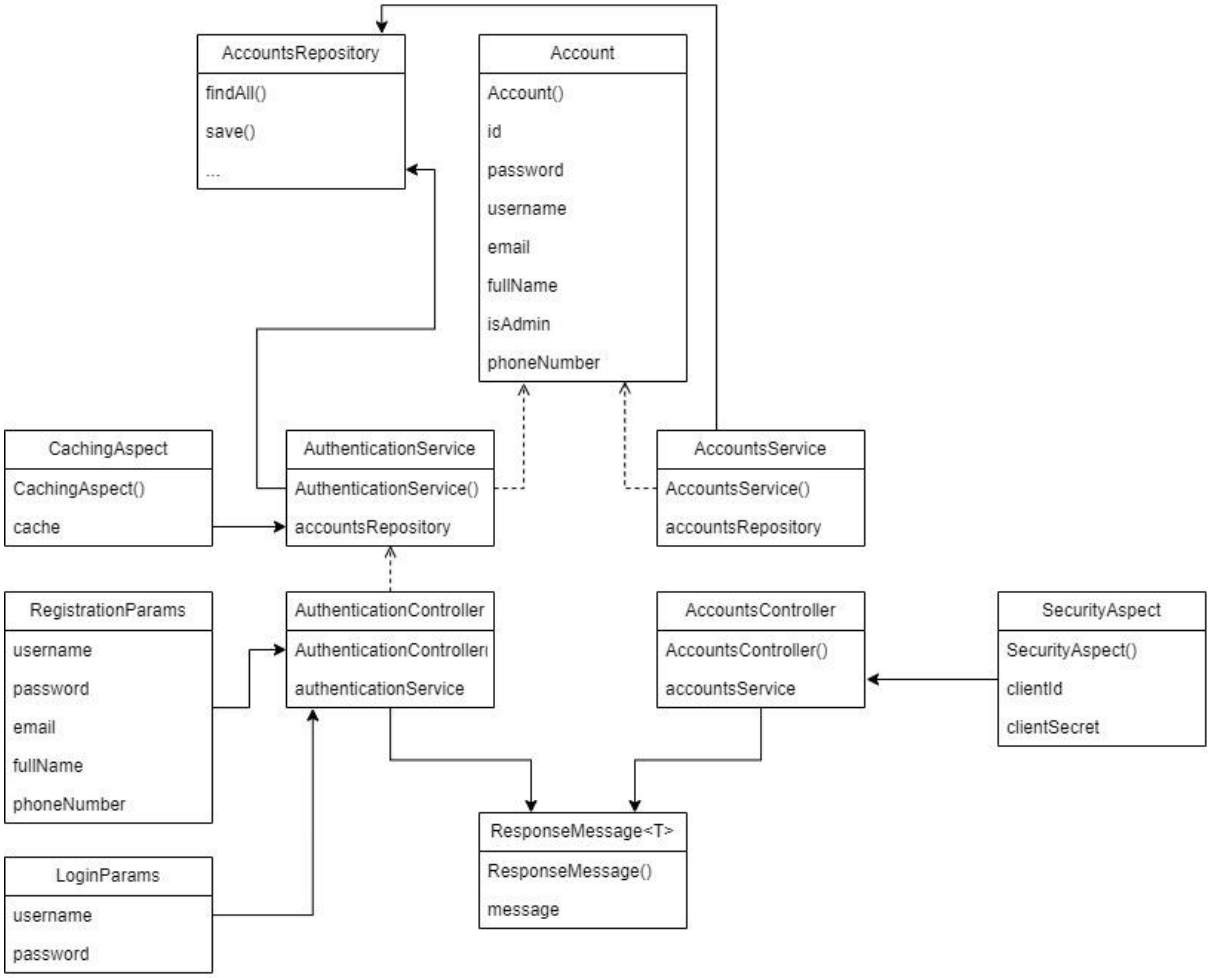
Entities:

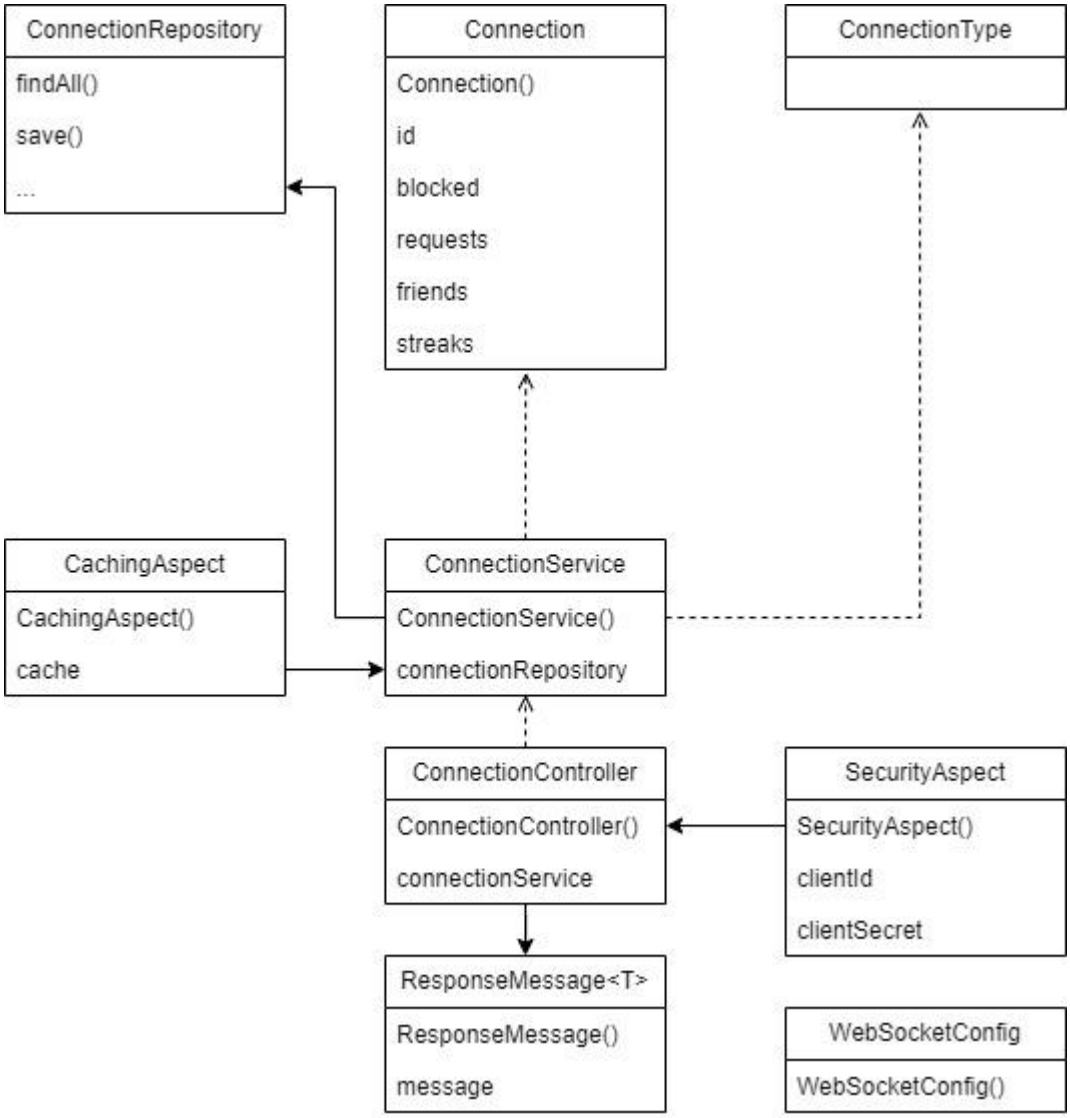
1. Account:
    - Attributes: Id, Username, Email, Password, ProfilePicture, Full Name, PhoneNumber, isAdmin
    - Relationships: Accounts can have connections (other Accounts), send and receive Messages, send and receive snaps and create stories.
2. Messages:
    - Attributes: Id, SenderId, ReceiverId, Content, Timestamp, Seen, Visibility
    - Relationships: Belongs to a Sender (Account) and Receiver (Account)
3. Connection:
    - Attributes: Id, Friends, Requests, Blocked, Streaks
    - Relationships: Belongs to a User (Account), and friends, requests, and blocked (Account)
4. Content:
    - Attributes: Id, Type, Url, PosterId, Timestamp, Seen, Receivers
    - Relationships: Belongs to a Poster(Account) and receivers (Account)

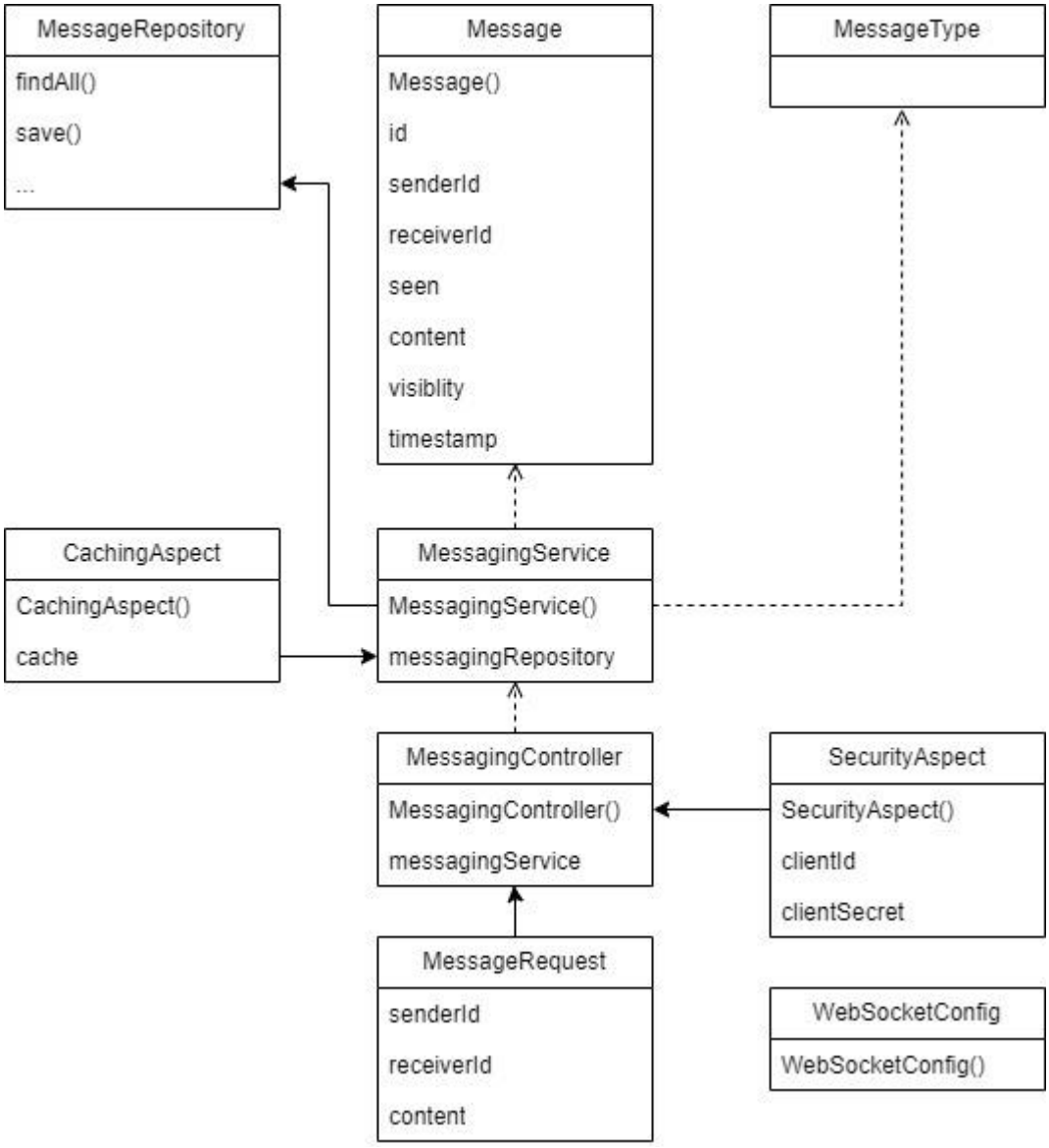
Relationships:

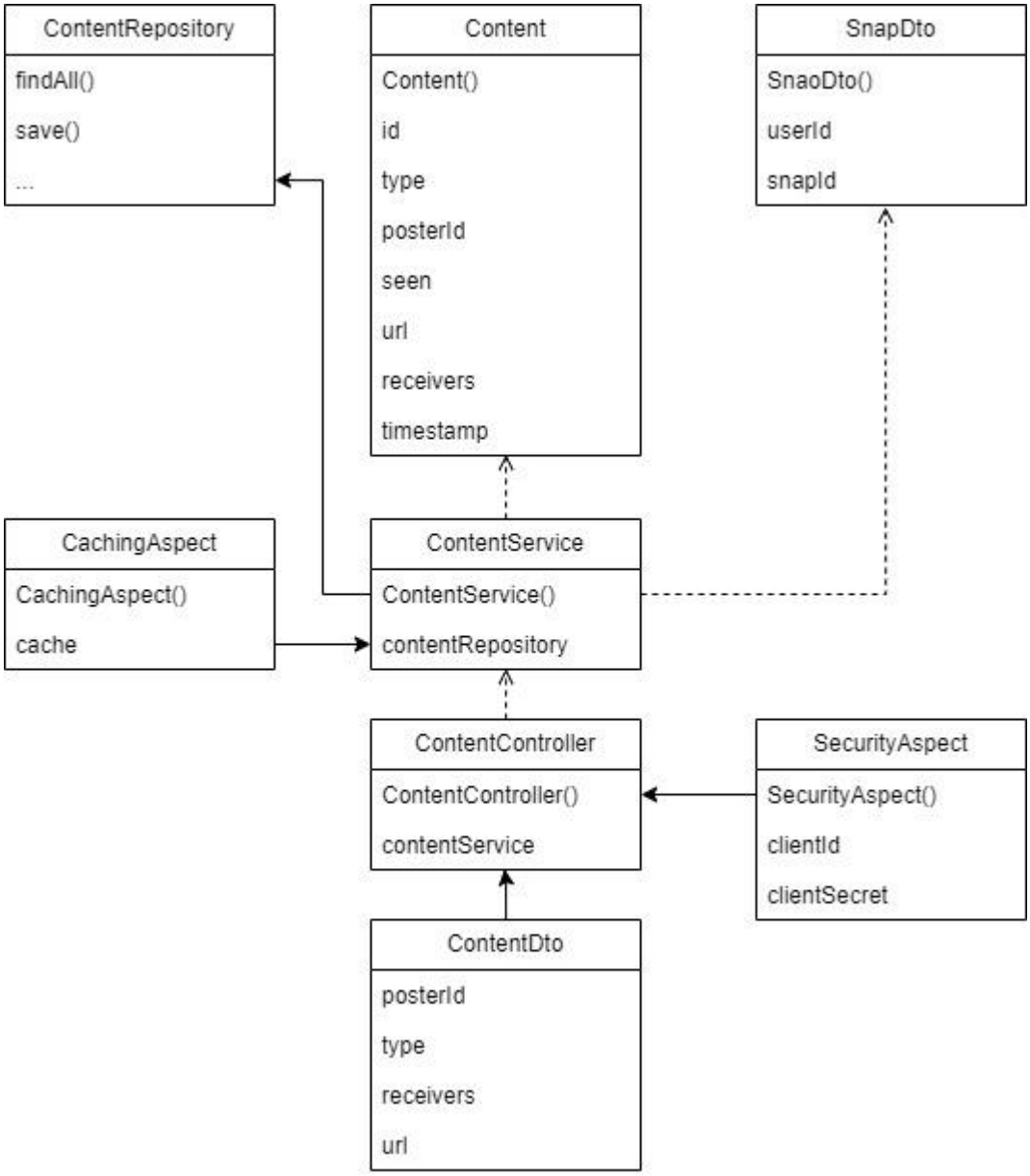
- Users and Friends: A many-to-many relationship. Users can have multiple friends and be friends with multiple users.
- Users and Messages: A one-to-many relationship. A user can send many messages but a message is sent by one user.
- Users and Connection: A one-to-one relationship. A user can have only one connection and a connection can only have one user.
- Users and Content: A one-to-many relationship. A user can create multiple stories/snaps, but a story/snap is created by one user.

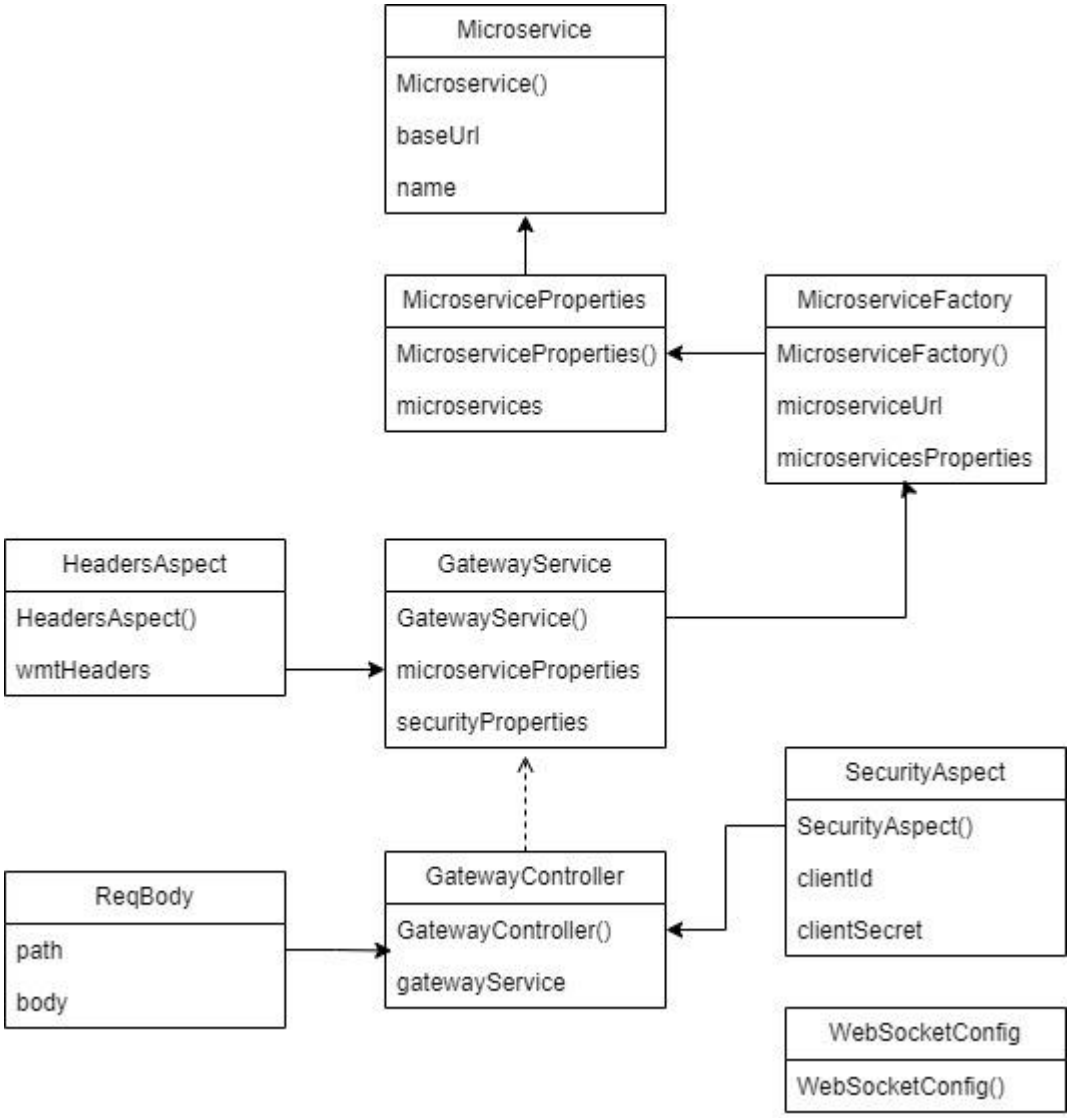
Conceptual Diagrams:




ConnectionRepository
findAll()
save()
...
Connection
Connection()
id
blocked
requests
friends
streaks
ConnectionType
CachingAspect
CachingAspect()
cache
ConnectionService
ConnectionService()
connectionRepository
ConnectionController
ConnectionController()
connectionService
SecurityAspect
SecurityAspect()
clientId
clientSecret
ResponseMessage<T>
ResponseMessage()
message
WebSocketConfig
WebSocketConfig()

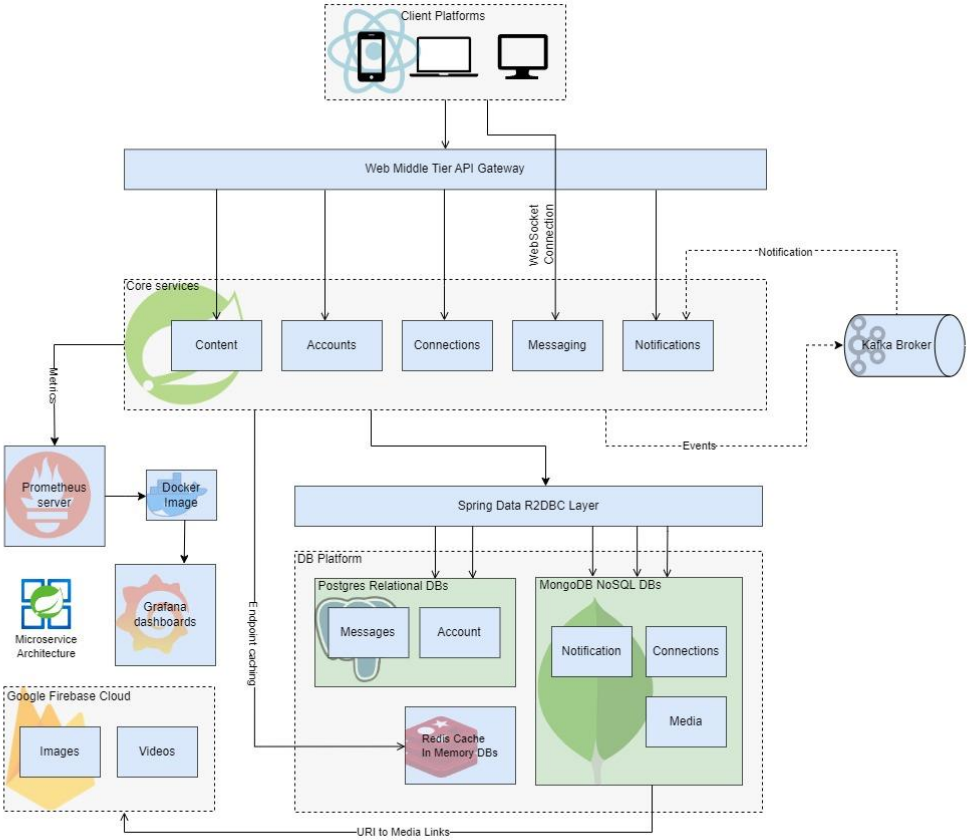

MessageRepository
findAll()
save()
...
Message
Message()
id
senderId
receiverId
seen
content
visiblity
timestamp
MessageType
CachingAspect
CachingAspect()
cache
MessagingService
MessagingService()
messagingRepository
MessagingController
MessagingController()
messagingService
SecurityAspect
SecurityAspect()
clientId
clientSecret
MessageRequest
senderId
receiverId
content
WebSocketConfig
WebSocketConfig()


ContentRepository
findAll()
save()
...
Content
Content()
id
type
posterId
seen
url
receivers
timestamp
SnapDto
SnaoDto()
userId
snapId
CachingAspect
CachingAspect()
cache
ContentService
ContentService()
contentRepository
ContentController
ContentController()
contentService
SecurityAspect
SecurityAspect()
clientId
clientSecret
ContentDto
posterId
type
receivers
url


Microservice
Microservice()
baseUrl
name
MicroserviceProperties
MicroserviceProperties()
microservices
MicroserviceFactory
MicroserviceFactory()
microserviceUrl
microservicesProperties
HeadersAspect
HeadersAspect()
wmtHeaders
GatewayService
GatewayService()
microserviceProperties
securityProperties
SecurityAspect
SecurityAspect()
clientId
clientSecret
ReqBody
path
body
GatewayController
GatewayController()
gatewayService
WebSocketConfig
WebSocketConfig()

Architectural Design

Conceptual Architecture:



Notes:

1. Client Platforms: Represent the various user interfaces through which users interact with the system, such as web applications, mobile apps, and desktop applications.
2. Web Middle Tier API Gateway: Acts as a single entry point for handling API requests from client platforms, routing them to appropriate microservices, and aggregating the results to return to the client.
3. Core Services:

- Content: Manages user-generated content like snaps, stories.
- Accounts: Handles user account information and authentication.
- Connections: Manages user connections, like friendships or blocked.
- Messaging: Deals with real-time and stored messaging between users.
- Notifications: Sends alerts or notifications to users about various events within the app.

4. Kafka Broker: Used for managing streaming data and handling events in real-time, facilitating a publish-subscribe messaging system.

5. Monitoring and Logging:

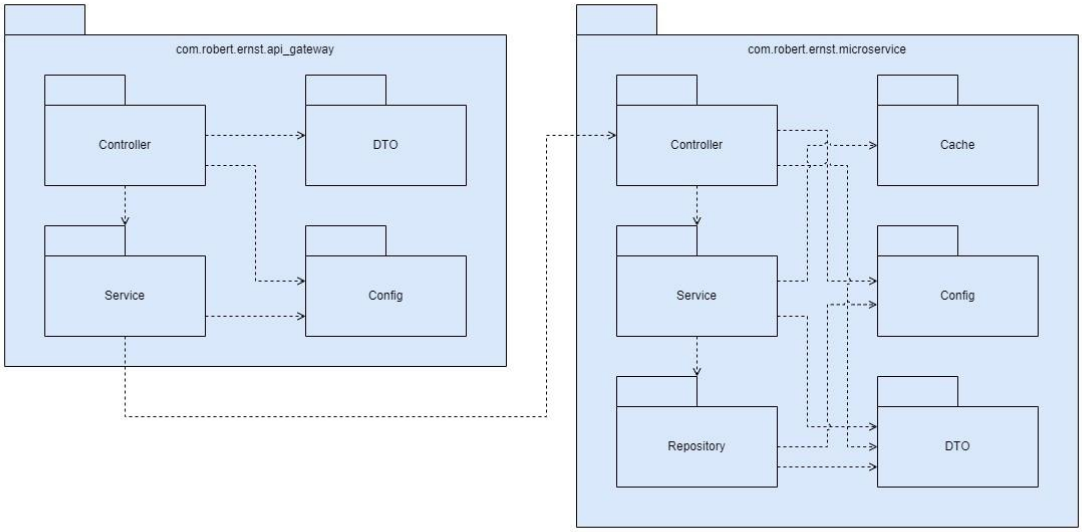
- Prometheus server: An open-source monitoring system with a time-series database used for recording real-time metrics.
- Grafana dashboards: Provides visualization and analytics for metrics collected from Prometheus, displaying them in a user-friendly dashboard.

6. DB Platform:

- Postgres Relational DBs: PostgreSQL databases used for structured data that requires complex queries with relationships, such as accounts and messages.
- MongoDB NoSQL DBs: NoSQL databases used for storing unstructured or semi-structured data, providing flexibility for schema-less data, likely used here for notifications and media content.
- Redis Cache: An in-memory data store used as a cache to improve the performance of the system by temporarily storing frequently accessed data.

7. Google Firebase Cloud: Storing and serving user-generated images and videos, leveraging Firebase's cloud storage and delivery capabilities.
8. Spring Data R2DBC Layer: Usage of the Spring Data R2DBC (Reactive Relational Database Connectivity) for database access in a non-blocking and reactive fashion, which is particularly suitable for scalable, cloud-native applications.

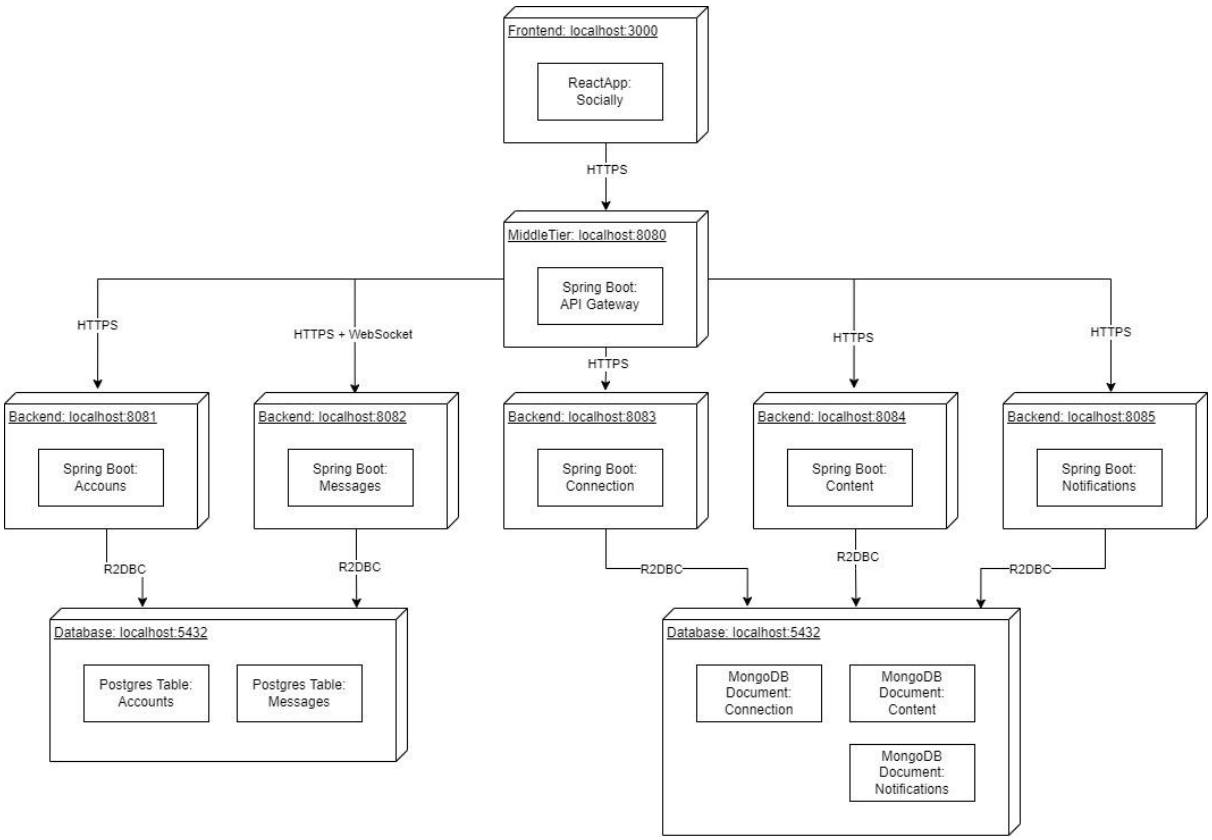
Package Design:



The microservice package structure is the same for all of Accounts, Connections, Content, Messaging and Notifications.

Component and Deployment Diagram

Deployment Diagram:



1. **Frontend**:
   ReactApp - Socially: The user interface of the application, running on localhost:3000. It's a React application providing a dynamic user experience. React being a single-page application framework, this front end would make API calls to the backend services to fetch and post data.
2. **Middle Tier**:
   Spring Boot - API Gateway: Located at localhost:8080, this is the entry point for all backend requests. The API Gateway is a Spring Boot application that routes incoming HTTPS requests to the appropriate microservices. It likely also handles cross-cutting concerns like authentication, logging, request tracing, etc.
3. **Backend Microservices**:

- Spring Boot - Accounts: Running on localhost:8081, this microservice manages user accounts. It connects to a PostgreSQL database on localhost:5432 and interacts with the "Accounts" table through R2DBC, which is a reactive relational database connectivity solution for Spring applications.

- Spring Boot - Messages: Hosted at localhost:8082, this microservice deals with messaging functionality, allowing for real-time communication through WebSockets in addition to standard HTTPS. It also uses R2DBC to interact with the "Messages" table in the PostgreSQL database.

- Spring Boot - Connection: This service runs on localhost:8083 and manages user connections, such as friends, blocked, etc. It uses R2DBC to interact with MongoDB documents labeled "Connection", indicating it handles semi-structured data that fits well with NoSQL databases.

- Spring Boot - Content: Available at localhost:8084, this microservice is responsible for handling user-generated content, stored in MongoDB with documents labeled "Content". R2DBC suggests that reactive database interactions are employed here as well.

- Spring Boot - Notifications: Running on localhost:8085, this service manages notifications for the app. It interacts with MongoDB documents labeled "Notifications", and like the others, it utilizes R2DBC for database interactions.

4. **Databases**:
- PostgreSQL: The relational database running on localhost:5432 with tables for "Accounts" and "Messages". This choice is used for transactional integrity and relational data management for those entities.

- MongoDB: A NoSQL database that stores documents for "Connection", "Content", and "Notifications". MongoDB is often used for its flexible schema, scalability, and performance with large volumes of data that do not fit a rigid relational model.

5. **Connections**:
- HTTPS is used as the primary protocol for secure communication between the frontend, the API Gateway, and the various microservices.

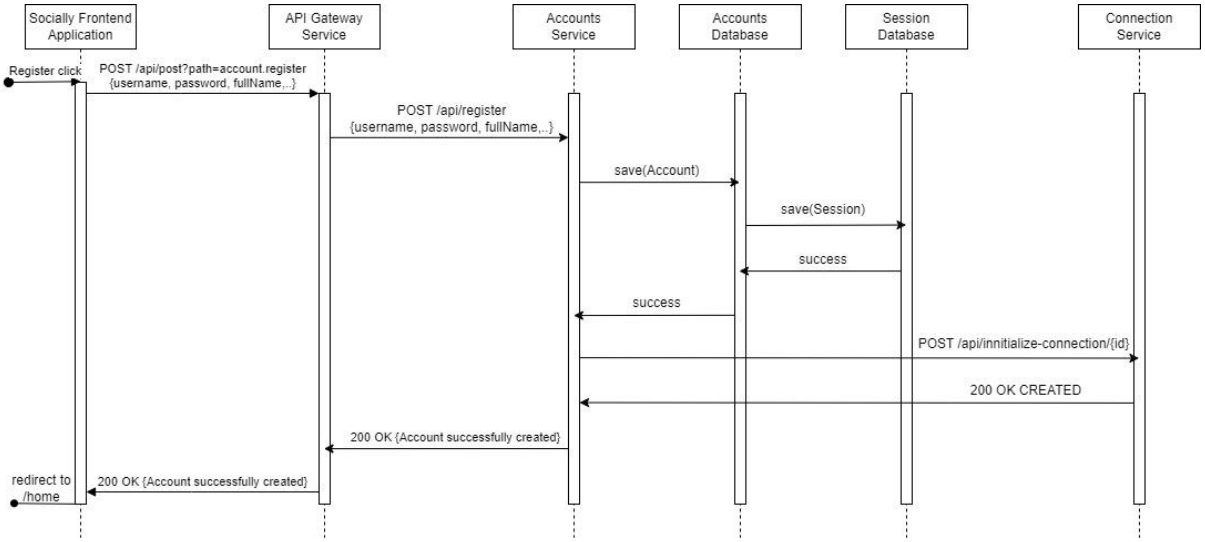
- HTTPS + WebSocket is specified between the frontend and the Messages service, suggesting that this service supports both regular secure HTTP requests and bidirectional WebSocket communication for real-time messaging.

Deliverable 3

Design Model

Dynamic Behavior

Register account scenario:

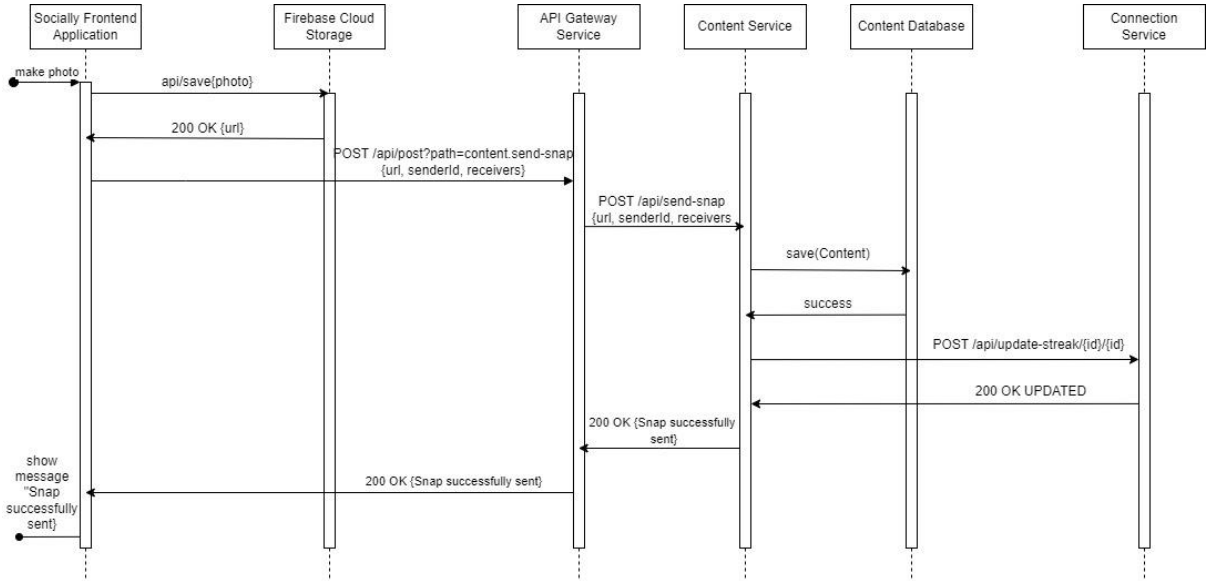


1. User Interaction:

- The user clicks on the register button in the Socially frontend application.

2. Account Creation:
    - The frontend sends a POST request to the API Gateway service (/api/post?path=account.register) with the new user's details (username, password, full name, etc.).
    - The API Gateway forwards this request to the Accounts service.
3. Accounts Service Processing:
    - The Accounts service processes the request and attempts to save the new account to the Accounts database.
    - Simultaneously, it saves a session to the Session database linked to the new account.
4. Initialize Connection:
    - After successfully creating the account and session, the Accounts service makes a call to the Connection service to initialize connection settings for the new account (/api/initialize-connection/{id}).
    - The Connection service initializes the settings and responds with 200 OK CREATED.
5. User Redirect:
    - The frontend receives the confirmation of account creation (200 OK) and redirects the user to the home page.

Send snap scenario:



1. User Interaction:
    - The user initiates the sequence by making a photo within the Socially *f*rontend *a*pplication.
2. Save Photo:
    - The photo is saved to Firebase Cloud Storage via the api/save(photo) endpoint.
    - Firebase Cloud Storage responds with a 200 OK status and a URL of the saved

photo.

3. Send Snap:
    - The frontend sends a POST request to the API Gateway Service (/api/post?path=content.send-snap) including the photo URL, sender ID, and receiver IDs.
    - The API Gateway forwards this request to the Content Service.
4. Content Service Processing:
    - The Content Service receives the snap details and makes a call to its internal API (/api/send-snap) to handle the snap.
    - It then saves the snap content to the Content Database.
    - After successfully saving the snap, the Content Service sends a response back indicating the snap was successfully sent (200 OK).
5. Update Connection Streak:
    - The Content Service calls the Connection Service to update the streak between users (/api/update-streak/{id}/{id}).
    - The Connection Service updates the streak and responds with a 200 OK UPDATED.
6. User feedback:
    - The frontend receives the confirmation of snap creation (200 OK) and then, the user is shown a “Snap successfully sent” toast.

Class Diagram

1. **Gateway Microservice**

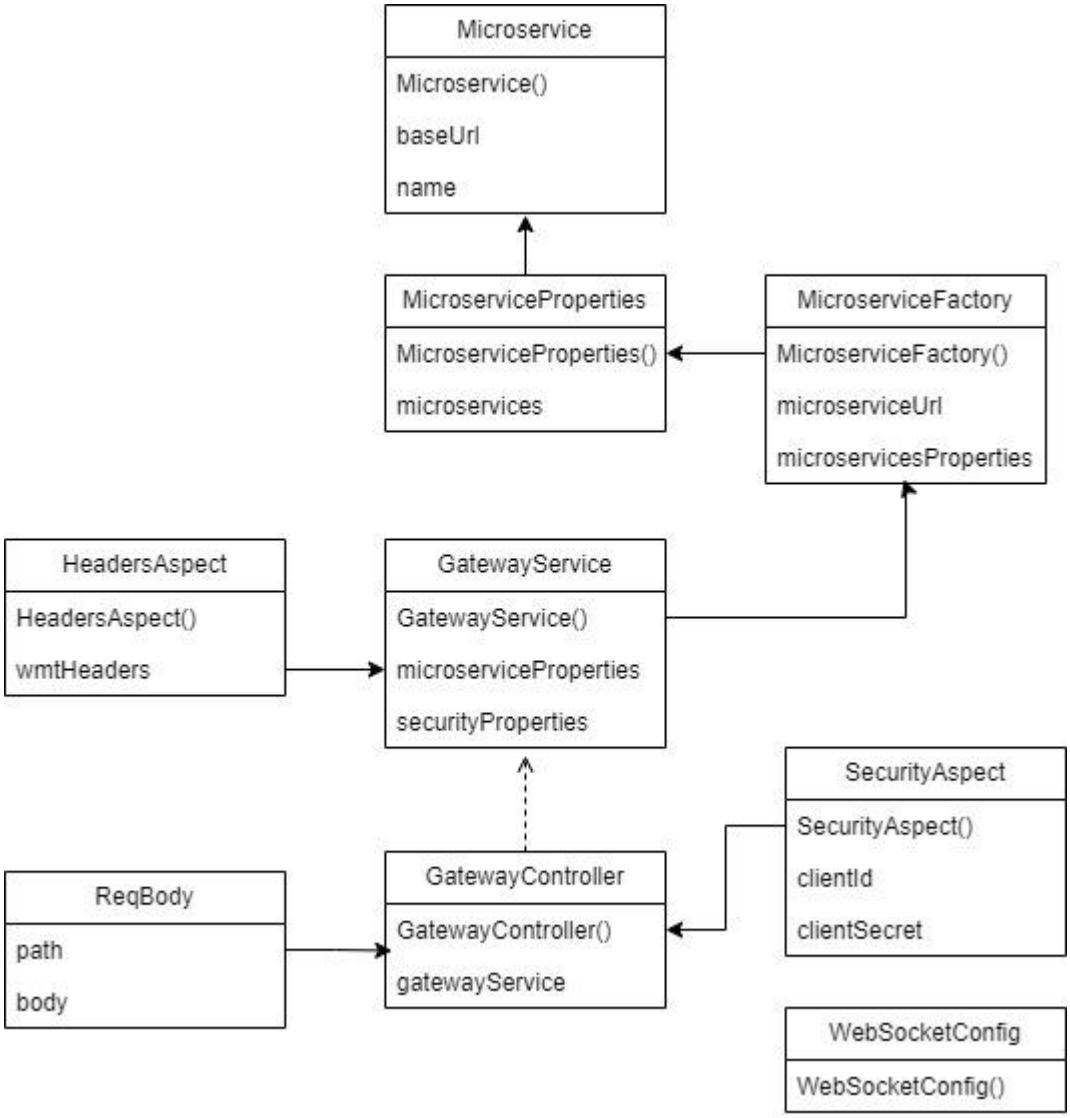


Diagram Components:

- Microservice, MicroserviceFactory, and MicroserviceProperties classes manage the configuration and creation of HttpClient instances, utilizing the Factory pattern for creating HttpClient objects dynamically based on yml configuration.
- GatewayService acts as a facade, simplifying the network requests towards various microservices by routing them accordingly.
- SecurityAspect and HeadersAspect implement cross-cutting concerns such as security and header manipulation using the Aspect-Oriented Programming (AOP) approach, closely related to the Decorator pattern in intent by allowing the attachment of additional responsibilities to objects.
- GatewayController directs incoming API requests to appropriate services, acting as part of a simplified Command pattern, where commands are the requests routed to handlers.

GoF Patterns:

- Factory Pattern: Used in MicroserviceFactory to encapsulate the logic of instantiating service endpoints, allowing for flexible service management and instantiation based on runtime conditions or configurations.
- Facade Pattern: GatewayService provides a unified interface to a set of interfaces in the microservices, making them easier to use.
- Decorator Pattern (AOP Style): Achieved through SecurityAspect and HeadersAspect, adding additional behavior to existing objects in a dynamic manner.

2. Accounts Microservice

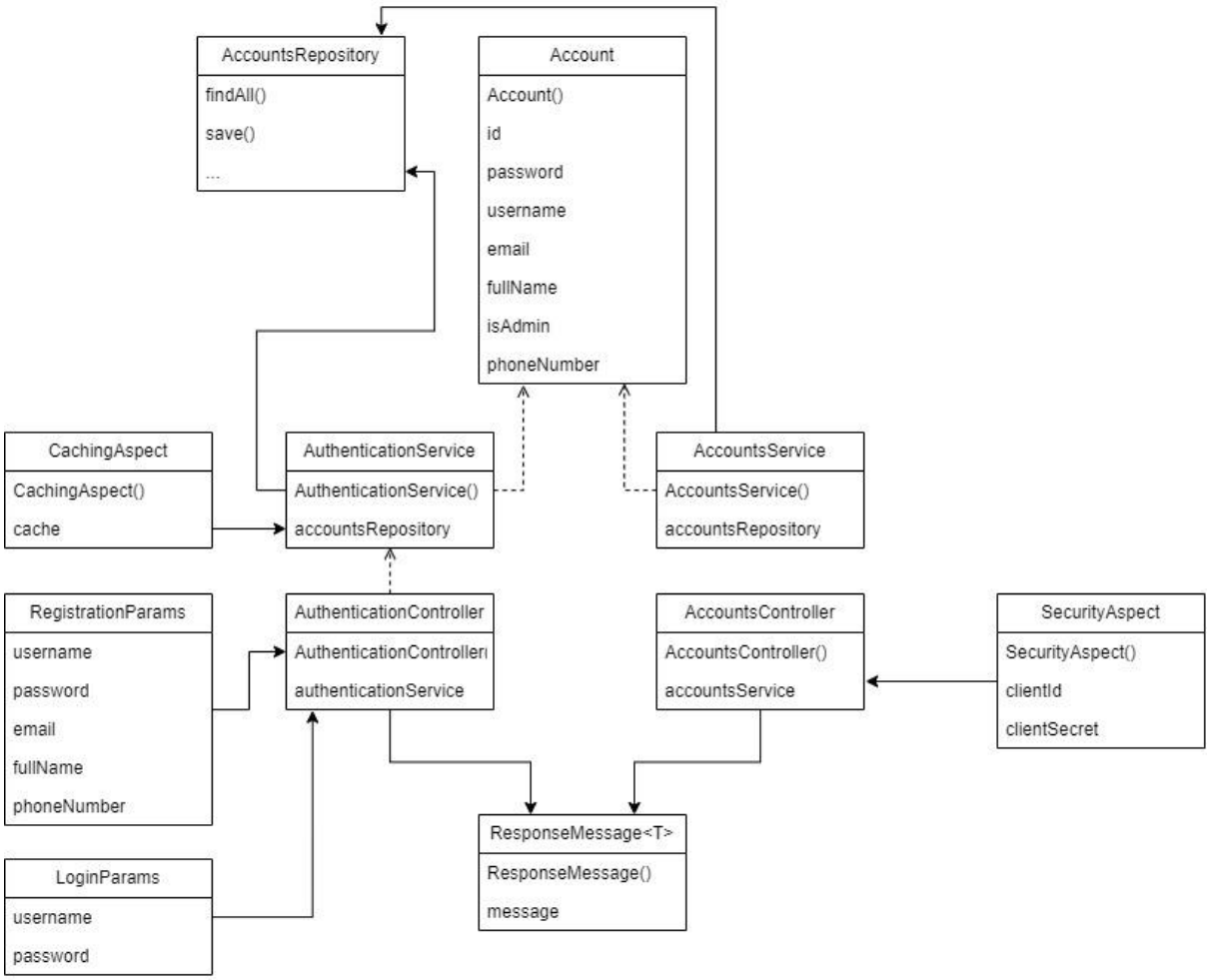


Diagram Components:

- AccountsRepository interacts with the database for CRUD operations.
- Account is a data model representing the user account.
- AccountsService encapsulates the business logic for account management.
- AccountsController handles HTTP requests relating to accounts.

GoF Patterns:

- Singleton Pattern: Likely used within AccountsService and AccountsRepository to ensure a single instance of each class handles all operations uniformly across the application.
- Strategy Pattern: Could be applied to encapsulate different algorithms or strategies for account validation or password encryption.

3. Connection Microservice

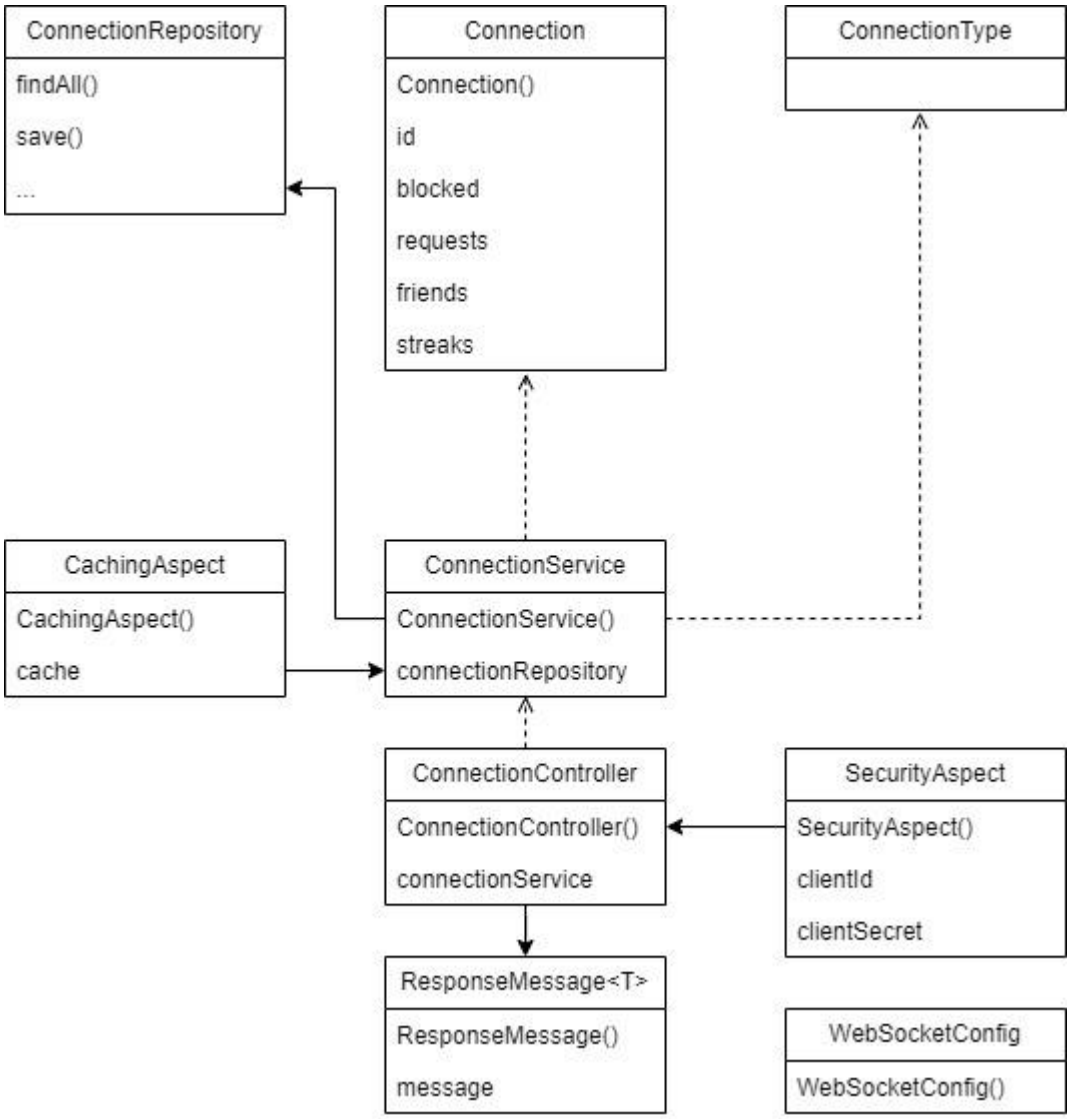


Diagram Components:

- ConnectionRepository and ConnectionService manage friends and connection-related data.
- Connection models the connection entities including friends and block lists.
- ConnectionController exposes endpoints for managing user connections.

GoF Patterns:

- Singleton Pattern: ConnectionService could be implemented as a singleton to ensure consistent state and behavior.
- Factory Pattern: Might be utilized for creating Connection instances based on different types of relationships or interactions (e.g., friend requests, block actions).

4. Messaging Microservice

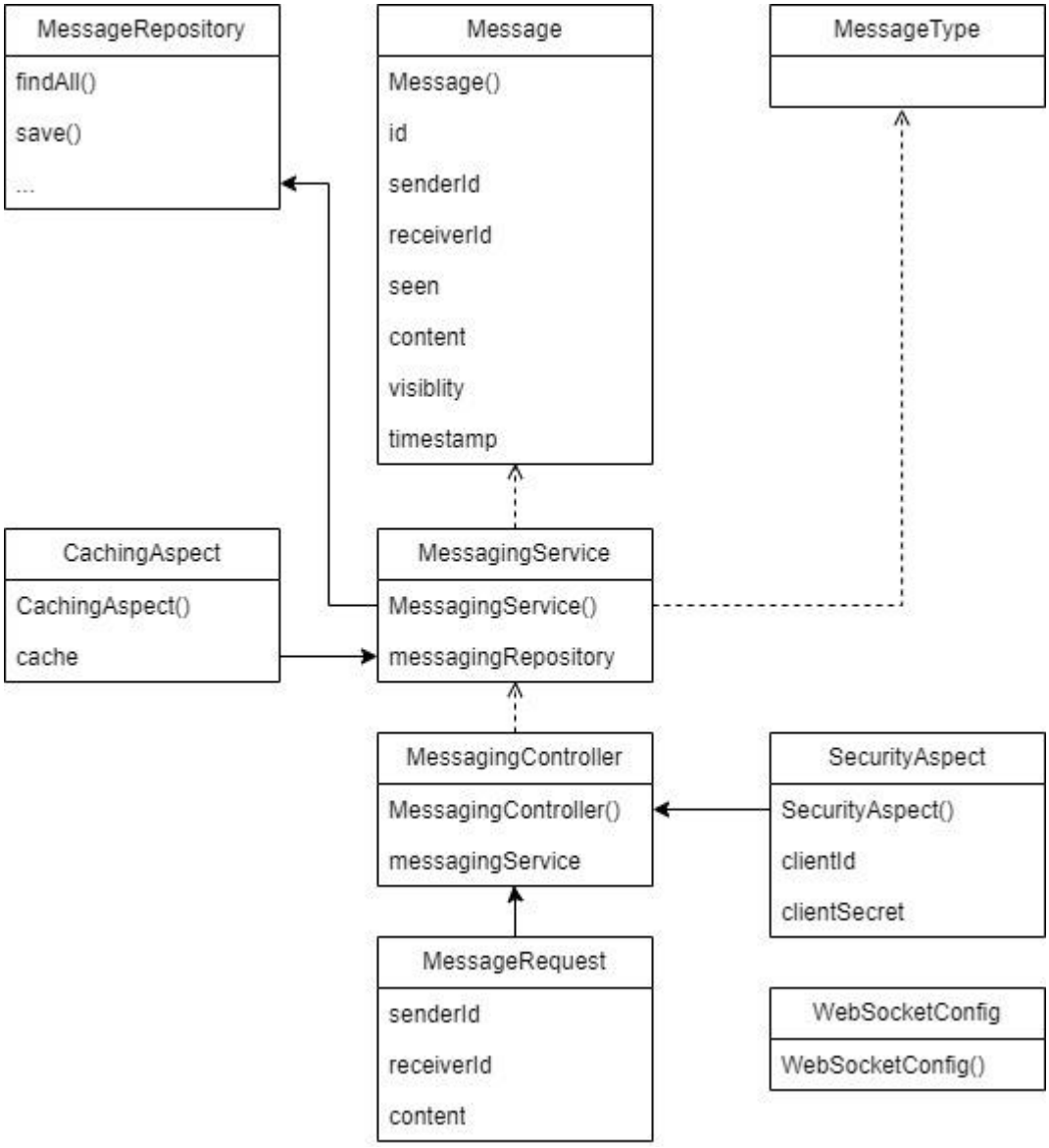


Diagram Components:

- MessageRepository and MessagingService manage message storage and business logic.
- Message is the data model for messages.
- MessagingController provides APIs for sending and retrieving messages.

GoF Patterns:

- Command Pattern: MessageRequest could be seen as a command object that encapsulates all information necessary to perform an action or trigger an event.
- Mediator Pattern: MessagingService acts as a mediator facilitating the interaction between various components (database, message processing logic, etc.).

5. Content Microservice

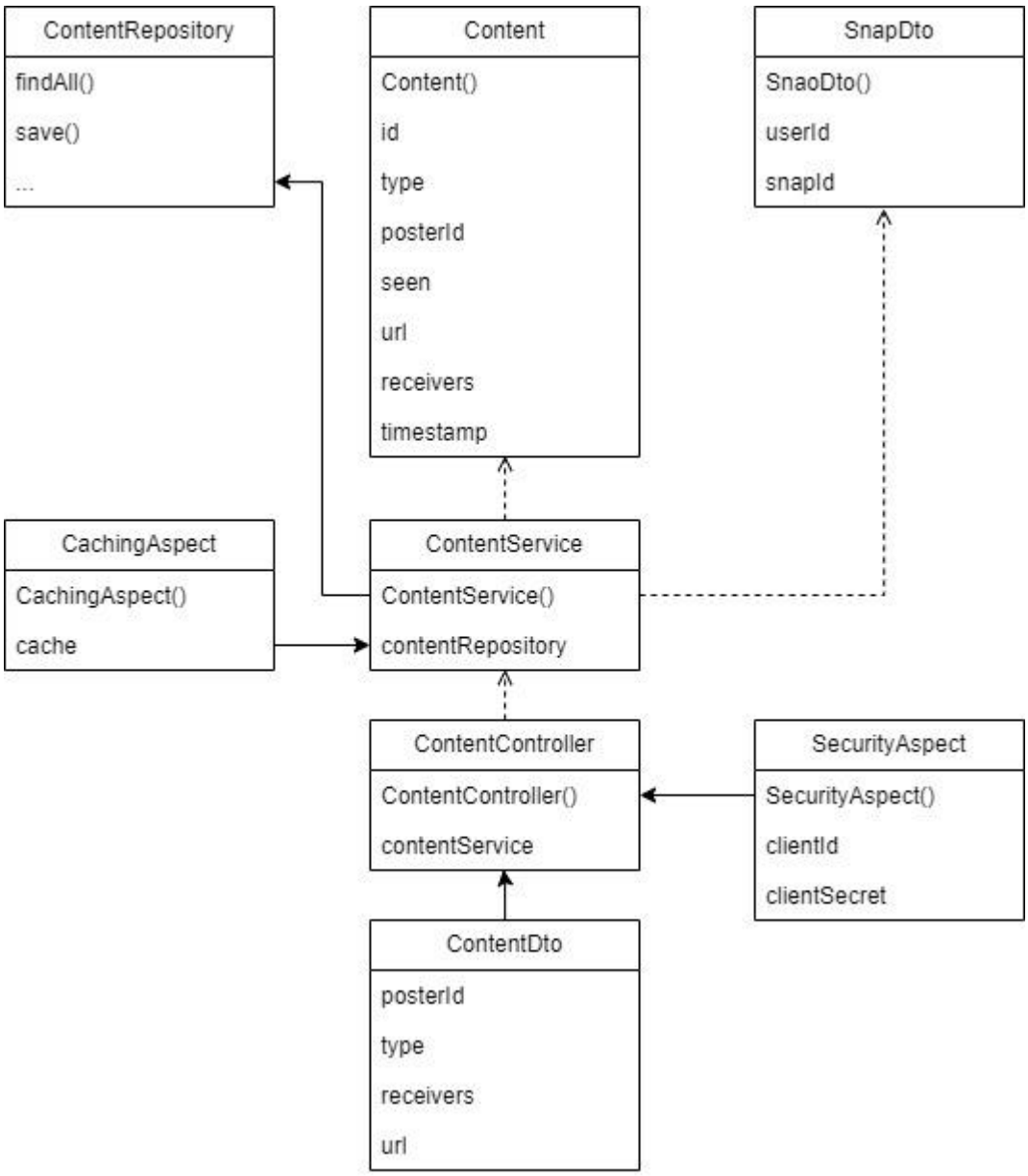


Diagram Components:

- ContentRepository and ContentService handle content-related operations.
- Content models the content data.
- ContentController serves endpoints for content management.

GoF Patterns:

- Observer Pattern: Useful in scenarios where content changes need to be propagated in real-time to subscribers, e.g., updates in content status or new content notifications.
- Strategy Pattern: Could be applied for different content handling strategies, such as different methods of content delivery or content storage optimization.

Conclusion

Each microservice is structured to effectively handle its part of the system's functionality, with clear separations of concerns and adherence to software design principles. Utilizing

GoF design patterns provides not only a robust architectural framework but also enhances the scalability, maintainability, and functionality of the microservices.

Data Model

Overview

This section outlines the data model and the interactions with the R2DBC database through reactive programming paradigms. I utilize Spring Data's ReactiveCrudRepository to enable non-blocking database access and efficient data management for a real-time application dealing with user interactions and media content management. I use Relational DB's for relational schemas like Accounts, Messages, and NoSQL documents for JSON alike schemas like Connections and Content.

Database Schema

The schema includes the following main entities:

1. Account (PostgreSQL Table):
    - id: Long
    - username: String
    - password: String
    - fullName: String
    - email: String
    - profilePhoto: String
    - phoneNumber: String
    - isAdmin: Boolean
2. Message (PostgreSQL Table)
    - id: Long
    - senderId: Long
    - receiverId: Long
    - content: String
    - timestamp: LocalDateTime
    - seen: Boolean
    - visibility: Long[]
3. Connection (MongoDB Document)
    - id: String
    - friends: User[]
    - blocked: User[]

- requests: User[]
- streak: Streak[]

4. Content (MongoDB Document)
    - id: String
    - type: String (SNAP|STORY)
    - url: String
    - posterId: Long
    - timestamp: LocalDateTime
    - seen: Long[]
    - receivers: Long[]
5. Session (PostgreSQL Table)
    - id: UUID
    - userId: Long
    - timestamp: LocalDateTime
    - deviceId: String
    - ipAddress: String
    - siteId: String

Usage Example

Demonstrates how to retrieve a list of snaps for a user reactively:

```
Flux<Snap> snaps = snapRepository.findByUserId(userId);
```

Conclusion

This model ensures efficient data handling and real-time operations essential for responsive user interfaces in our application, leveraging the full capabilities of reactive programming with R2DBC and Spring Data.

System Testing

Overview

This document details the system testing methodologies applied to ensure the robustness and reliability of the account management module within our application. We employ a variety of integration and reactive testing strategies to verify the functionality and performance of the application components under real-world usage scenarios.

Testing Methodologies

- Integration Testing:

Purpose: To validate the interactions among various application components including controllers, services, and repositories.

Tool: Utilizes Spring Boot's @SpringBootTest with WebTestClient for making non-blocking calls to REST endpoints in a context that simulates running application.

- Reactive Testing:

Purpose: To ensure that reactive data streams are managed correctly across the system, testing both the assembly and execution of reactive chains.

Tool: Project Reactor's StepVerifier is used to create test sequences that verify reactive flows and ensure expected behaviors.

- Automated API Testing:

Purpose: To confirm that RESTful endpoints function correctly across various scenarios including handling of errors, performance, and consistency.

Tool: WebTestClient is deployed to simulate requests to the controllers and to test endpoint responsiveness directly.

- Security Testing:

Purpose: Tests all endpoints to enforce security protocols, manage authentication, and protect against vulnerabilities.

Tool: Spring Security Test supports mocking of security contexts and asserting authentication within tests.

Example Test Case:

Get Account by ID

- **Success Case**: Validate successful retrieval of account details when provided a valid user ID. Ensure HTTP 200 OK and that returned data matches the expected account details.

- **Failure Case**: Test the API's response when an account ID does not exist.

Expected response is HTTP 404 NOT FOUND, indicating no account found.

Additional Resources

- Postman Collections: Each microservice includes a Postman collection located in its respective docs/postman directory. These collections contain pre-configured requests that can be used for:
- Manual Testing: Facilitates exploratory testing by developers or QA personnel.
- Development: Assists developers in simulating client requests during feature

development.

- Automation: Supports automated API tests and integration scenarios.

The Postman collections are regularly updated to reflect the current API specifications and include comprehensive documentation on how to execute each request. They serve as a valuable tool for understanding and interacting with the microservices independently of the system tests.

Future Improvements

Caching Migration to Redis

- Current State: The current application uses custom aspects and hashmaps to manage caching. While this approach provides a degree of control over cache behavior, it has limitations in scalability, persistence, and management across distributed systems.

- Future Improvement: Migrate the caching layer to Redis, a robust, open-source, in-memory data structure store, used as a database, cache, and message broker.

- Benefits:

- **Scalability**: Redis offers built-in mechanisms to scale out through clustering and provides high availability with Redis Sentinel.
- **Performance**: Being an in-memory data store, Redis dramatically reduces the data access latency compared to traditional disk-based databases.
- **Persistence**: Redis supports various persistence options to ensure that data is not lost during failures.
- **Data Structures**: Redis supports diverse data structures such as strings, hashes, lists, sets, sorted sets with range queries which can be leveraged to optimize data storage and retrieval.
- **Simplified Management**: Using Redis can simplify the management of caching logic, especially in distributed environments, by handling data consistency and replication across instances.

Kafka for Notifications

- Current State: Notifications are currently managed via WebSockets and Server-Sent Events (SSE), which are suitable for real-time communication but may not efficiently handle high-throughput scenarios or guarantee message delivery in complex distributed systems.

- Future Improvement: Implement Apache Kafka for handling notification services. Kafka is a distributed event streaming platform capable of handling trillions of events a day.

- Benefits:

- **Scalability**: Kafka's distributed nature allows it to scale out without downtime, handling more messages as demand increases.

- **Reliability**: Kafka ensures data is replicated and persisted to disk, guaranteeing no data loss.
- **High Throughput**: Kafka can handle thousands of messages per second, supporting high-volume event processing.
- **Decoupling of System Components**: Using Kafka allows decoupling of data streams and system components, making the system more robust to changes and easier to manage.
- **Versatile Integration**: Kafka integrates with a variety of streaming data processing tools like Apache Flink and Apache Spark, as well as with traditional batch processing systems, providing flexibility in how data is consumed and processed.

Build and Deployment Optimization

- Current State: The build and deployment processes may be manually managed or not fully optimized for the best continuous integration/continuous deployment (CI/CD) practices.

- Future Improvement: Streamline Build and Deployment processes using modern DevOps tools and practices.

- Benefits:

- Automation: Utilizing CI/CD pipelines (e.g., Jenkins, GitLab CI/CD, GitHub Actions) for automated testing, building, and deployment of applications.
- Containerization: Adopting containerization with Docker and orchestration with Kubernetes to manage application deployments in a scalable and isolated manner.
- Configuration Management: Implementing tools like Ansible, Chef, or Puppet to automate the configuration of servers and applications.
- Monitoring and Logging: Integrating comprehensive monitoring and logging solutions (e.g., Prometheus, Grafana, ELK Stack) to proactively manage application health and diagnose issues.
- Faster Release Cycles: Reducing the time from development to deployment through automation, allowing for faster iteration and feedback.

Improvements Conclusion

Implementing these future improvements will not only address current limitations but also prepare the application for scalable growth, enhanced performance, and improved reliability. By modernizing the caching system, notification handling, and deployment strategies, the application can maintain a competitive edge and adapt more readily to changing business requirements.

Conclusion

This documentation has thoroughly outlined the design, implementation, and testing frameworks used in this university project, demonstrating how modern software engineering principles and reactive programming paradigms can be effectively applied in a real-world simulation. By integrating advanced technologies such as R2DBC, WebTestClient, and frameworks like Spring Boot, the project not only meets its functional requirements but also offers insights into the complexities of building scalable, responsive web applications.

The adaptation of Redis for enhanced caching mechanisms, the use of Apache Kafka for robust notification systems, and the implementation of sophisticated CI/CD pipelines represent a forward-thinking approach to handling data-intensive operations in a distributed environment. These enhancements are proposed not just as upgrades to the system's capabilities, but as opportunities for further research and exploration into distributed systems, performance optimization, and automated deployment strategies.

As this project is part of an academic endeavor, it provides a solid foundation for future projects and dissertations. The experiences gained and the challenges encountered during this project contribute significantly to our understanding of both theoretical and practical aspects of system architecture and software development.

Moving forward, it is encouraged for future students and researchers to build upon this work, explore the proposed improvements, and continuously adapt the project to incorporate emerging technologies and methodologies. This will not only maintain the project's relevance in the rapidly evolving tech landscape but also enhance the educational value for subsequent cohorts of students.

This documentation is intended to serve as both a blueprint and a learning tool, inspiring further innovation and inquiry in the field of software engineering and data systems. Your feedback and contributions are highly valued as they will enhance the educational impact and technical robustness of this ongoing academic project.

Bibliography

https://www.baeldung.com/
https://spring.io/guides
https://app.diagrams.net/
https://stackoverflow.com/
https://react.dev/
https://www.mongodb.com/lp/cloud/atlas/try4?utm_content=controlhterms&utm_source=google&utm_campaign=search_gs_pl_evergreen_atlas_core_prosp-brand_gic-null_emea-ro_ps-all_desktop_eng_lead&utm_term=mongodb&utm_medium=cpc_paid_search&utm_ad=e&utm_ad_campaign_id=12212624554&adgroup=115749711503&cq_cmp=12212624554&gad_source=1&gclid=Cj0KCQjwxqayBhDFARIsAANWRnTj6IuxossiW5RHubF8UQBE1oombzNdAVy9EKqAmZ_aHi8nc0BVW2gaAhJIEALw_wcB
https://medium.com/
https://firebase.google.com/docs